



## Output Window



## Compilation Results

Custom Input

## Compilation Completed

• Case 1

Input:

head =

1 3 4

x =

3

Your Output:

1 &lt;-&gt; 3

Expected Output:

1 &lt;-&gt; 3

6 Node prev;

7

8 Node(int val) {

9 data = val;

10 next = null;

11 prev = null;

12 }

13 }

14 \*/

15 class Solution {

16 public Node delPos(Node head, int x) {

17

18 if (head == null)

19 return null;

20

21 if (x == 1) {

22 head = head.next;

23 if (head != null)

24 head.prev = null;

25 return head;

26 }

27

28 Node temp = head;

29

30 for (int i = 1; i &lt; x; i++) {

31 temp = temp.next;

32 }

33

34 temp.prev.next = temp.next;

35

36 if (temp.next != null) {

37 temp.next.prev = temp.prev;

38 }

39

40 return head;

41 }

42 }



Custom Input

Compile &amp; Run

Submit

[Courses](#) ▾[Tutorials](#) ▾[Practice](#) ▾[Jobs](#) ▾[Problem](#)[Editorial](#)[Submissions](#)[Comments](#)

# Output Window

[Compilation Results](#)[Custom Input](#)

## Compilation Completed

[Case 1](#)

Input:

k =

1

head =

2 4 6 7 5 2

Your Output:

true

Expected Output:

true

Java (21) ▾

[Start Timer](#)

```
1  /*
2  class Node
3  {
4      int data;
5      Node next;
6      Node(int d)
7      {
8          data = d;
9          next = null;
10     }
11 }
12 */
13 class Solution {
14     boolean isCircular(Node head) {
15         if (head == null)
16             return true;
17         Node temp = head.next;
18         while (temp != null && temp != head) {
19             temp = temp.next;
20         }
21         if (temp == head)
22             return true;
23         return false;
24     }
25 }
```

[Custom Input](#)[Compile & Run](#)[Submit](#)

## Compilation Completed

• Case 1

Input:  

head =

10 7 8 4 6

key =

8

Your Output:

10 -&gt; 7 -&gt; 4 -&gt; 6

Expected Output:

10 -&gt; 7 -&gt; 4 -&gt; 6

```
6 Node(int x) {
7     data = x;
8     next = null;
9 }
10 }
11 */
12
13 class Solution {
14     public Node deleteNode(Node head, int key) {
15         if (head == null)
16             return null;
17         if (head.data == key) {
18             if (head.next == head)
19                 return null;
20             Node last = head;
21             while (last.next != head) {
22                 last = last.next;
23             }
24             last.next = head.next;
25             head = head.next;
26             return head;
27         }
28         Node prev = head;
29         Node temp = head.next;
30         while (temp != head) {
31             if (temp.data == key) {
32                 prev.next = temp.next;
33                 return head;
34             }
35             prev = temp;
36             temp = temp.next;
37         }
38         return head;
39     }
40 }
41 }
42 }
```

[Custom Input](#)

Compile &amp; Run

Submit

## Output Window

Compilation Results

Custom Input

### Compilation Completed

• Case 1

Input:  

head =

1 7 8 10

key =

5

Your Output:

5 -> 1 -> 7 -> 8 -> 10

Expected Output:

5 -> 1 -> 7 -> 8 -> 10

```
1  /*
2  class Node {
3      int data;
4      Node next;
5
6      Node(int x) {
7          data = x;
8          next = null;
9      }
10 }
11 */
12 class Solution {
13     public Node insertAtBeginning(Node tail, int key) {
14
15         Node newNode = new Node(key);
16
17         if (tail == null) {
18             newNode.next = newNode;
19             return newNode;
20         }
21
22         newNode.next = tail.next;
23         tail.next = newNode;
24
25         return tail;
26     }
27 }
```



Custom Input

Compile & Run

S



## Compilation Completed

• Case 1

Input:  

head =

2 4 5

p =

2

x =

6

Your Output:

2 &lt;-&gt; 4 &lt;-&gt; 5 &lt;-&gt; 6

Expected Output:

2 &lt;-&gt; 4 &lt;-&gt; 5 &lt;-&gt; 6

```
1  /*
2  class Node
3  {
4      int data;
5      Node next;
6      Node prev;
7      Node(int data)
8      {
9          this.data = data;
10         next = prev = null;
11     }
12 }
13 */
14
15 class Solution {
16     Node insertAtPos(Node head, int pos, int data) {
17
18         Node newNode = new Node(data);
19
20         Node temp = head;
21
22         for (int i = 0; i < pos; i++) {
23             temp = temp.next;
24         }
25
26         newNode.next = temp.next;
27         newNode.prev = temp;
28
29         if (temp.next != null) {
30             temp.next.prev = newNode;
31         }
32
33         temp.next = newNode;
34
35         return head;
36     }
37 }
```

[Custom Input](#)

Compile &amp; Run

Submit

# 1. Two Sum

Solved 

Easy

 Topics Companies Hint

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

## Example 1:

**Input:** `nums = [2,7,11,15]`, `target = 9`

**Output:** `[0,1]`

**Explanation:** Because `nums[0] + nums[1] == 9`, we return `[0,1]`.

## Example 2:

**Input:** `nums = [3,2,4]`, `target = 6`

**Output:** `[1,2]`

## Example 3:

**Input:** `nums = [3,3]`, `target = 6`

**Output:** `[0,1]`

## Constraints:

- `2 <= nums.length <= 104`

 69.2K

2K



## </> Code

Java   Auto

```
1 class Solution {
2     public int[] twoSum(int[] nums, int target) {
3         HashMap<Integer, Integer> map = new HashMap<>();
4
5         for (int i = 0; i < nums.length; i++) {
6             int x = target - nums[i];
7
8             if (map.containsKey(x)) {
9                 return new int[]{map.get(x), i};
10            }
11
12            map.put(nums[i], i);
13        }
14
15        return new int[]{};
16    }
17 }
```

Saved

Ln 1, Col 1

☒ Testcase |  Test Result**Accepted** Runtime: 0 ms☒ Case 1☒ Case 2☒ Case 3

Input

`nums =``[2,7,11,15]`